

BEL C22 · JAVA TRACK

Java Revision Study Guide

Condensed notes, code snippets, mental models & memory tricks
for the full course — one doc instead of 60+ files.

C1 Java Fundamentals

C2 JVM Internals

C3 Packages, Encapsulation & Access

C4 Inheritance & Polymorphism

C5 Abstraction & Object Relationships

C6 Design Principles & SOLID

C8 Applying OOP Relationships

C9 Creational Design Patterns

— Structural & Behavioral Patterns

App Practice Problems Index

Class 1 — Java Fundamentals

Syntax and building blocks: the vocabulary you need before OOP makes sense.

• Intro to Java

- High-level, OO, "write once run anywhere."
- Pipeline: `.java` → `javac` → `.class` bytecode → JVM → machine code.
- JDK ⊃ JRE ⊃ JVM — develop ⊃ run ⊃ execute.

• Variables & Scope

- **Local** (in a method, no default, must init), **instance** (per object, defaults), **static** (shared, one copy).
- Memory: local → stack, instance → heap, static → method area.

• Operators

- Arithmetic `+` `-` `*` `/` `%`, relational `==` `!=` `>` `<`, logical `&&` `||` `!`, assignment `+=` `-=`.
- `10 / 3 = 3` — int division truncates.
- `x++` use-then-add vs `++x` add-then-use.

• Loops & Jumps

- **for**: known count. **while**: check first. **do-while**: runs ≥1 time (checks after).
- `break` exits loop, `continue` skips iteration, `return` exits method.

• Data Types

- 8 primitives (byte/short/int/long/float/double/char/boolean) — hold values directly, never null.
- Reference types (String, arrays, objects) hold an address, default `null`.
- **Trick**: primitive = box with the value; reference = label pointing at the box.

• Type Casting

- **Widening** (small → big): automatic, no data loss.
`byte` → `short` → `int` → `long` → `float` → `double`
- **Narrowing** (big → small): explicit `(int) d`, can lose data.
- **Trick**: "widen for free, narrow on purpose."

• Control Flow

- `if/else-if` for ranges & complex conditions.
- `switch` for exact-value matching — don't forget `break`.

• Methods & Constructors

- Static method → call via `ClassName.m()`; instance method → via `obj.m()`.
- Constructor: no return type, same name as class, runs on `new`.

C1 OOP Introduction — the 4 Pillars

IN SHORT

- OOP models real things as **objects**: a **class** is the blueprint, an **object** is the instance.
- **Encapsulation** — bundle data+methods, hide internals (private fields, public getters/setters).
- **Inheritance** — a class acquires fields/methods from a parent (extends).
- **Polymorphism** — same method name, different behavior per object.
- **Abstraction** — hide implementation, expose only what's necessary.

MENTAL MODEL — THE 4 PILLARS, ONE ANALOGY EACH

Encapsulation
ATM: buttons, not circuits

Inheritance
Dog extends Animal

Polymorphism
shape.draw() varies

Abstraction
drive without engine internals

Trick: mnemonic — "Every Inherited Pillar Abstracts" (E-I-P-A).

Practice (s10): Airtribe Learner Manager — console CRUD app using parallel arrays, Scanner, switch, age-validation with if-else, average via a while-driven menu loop.

Class 2 — JVM Internals

What actually happens between `javac` and your program running.

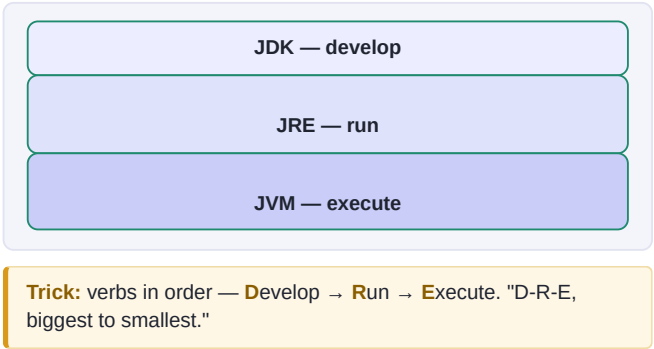
C2 JDK, JRE & JVM

IN SHORT

- **JDK** = build & run (`javac`, `javadoc`, `jar` + JRE inside). Developers install this.
- **JRE** = run only (JVM + core libraries). No compiler.
- **JVM** = executes bytecode; gives platform independence, does GC.

Role	Needs
Developer	JDK
End user	JRE

MENTAL MODEL — NESTING DOLLS

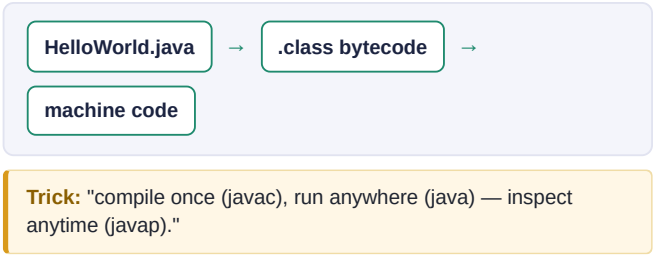


C2 Compilation & Bytecode

IN SHORT

- Bytecode = platform-independent intermediate form, not human-readable.
- `javac X.java → X.class`; `java X` runs it; `javap -c X.class` disassembles it.
- Bytecode is **verified** before execution — a security layer.

MENTAL MODEL — THE PIPELINE

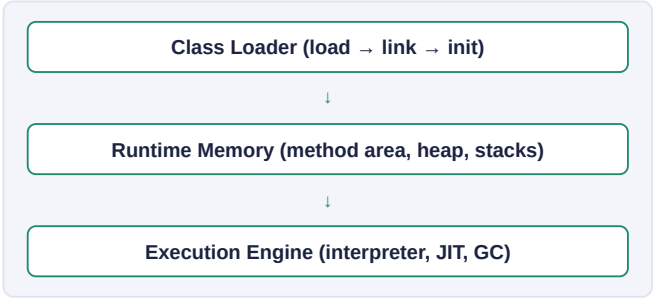


C2 JVM Architecture

IN SHORT

- 4 blocks: **Class Loader** → **Runtime Memory** → **Execution Engine** → **Native Interface (JNI)**.
- Execution Engine = Interpreter + JIT Compiler + Garbage Collector.

MENTAL MODEL — THE ASSEMBLY LINE



C2 Class Loading

IN SHORT

- 3 loaders: **Bootstrap** (core java.*) → **Platform/Extension** → **Application** (your classes).
- Parent delegation**: a loader always asks its parent first; only loads it itself if the parent can't. Stops user code from shadowing core Java classes.
- 3 sub-steps overall: Loading → Linking → Initialization.

MENTAL MODEL — ASK YOUR PARENT FIRST

Bootstrap (java.lang, java.util)
| Platform / Extension
| Application (your classes)

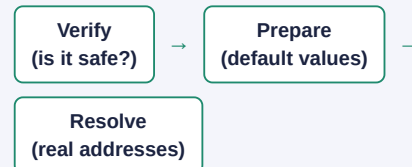
Trick: request flows UP to Bootstrap first, loading happens on the way back DOWN only if nobody above could do it.

C2 Linking Phase

IN SHORT

- Verification** — is the .class file safe & well-formed (magic number, type safety)?
- Preparation** — allocate static vars, set them to *default* values (0/false/null) — not your values yet.
- Resolution** — convert symbolic references (class names as strings) into real memory addresses.

MENTAL MODEL — 3 CHECKPOINTS



Trick: "V-P-R." static int count=10; → is 0 after Prepare, 10 only after Initialization.

C2 Initialization

IN SHORT

- Final class-loading step: static vars get their *real* values, static blocks run — top to bottom, in source order.
- Superclass initializes before subclass. Static blocks run **once ever**, can't touch instance vars.
- Triggers: new, accessing a static field/method, Class.forName().

SNIPPET

```
static int x = 10;           // step 1
static { x = 20; }           // step 2 (block runs)
static int y = x + 5;         // step 3: y = 25
```

Trick: "top-to-bottom, once, parent-before-child."

C2 Runtime Memory Areas

IN SHORT

Area	Shared?	Stores	Fails with
Method Area	Yes	class metadata, statics	OutOfMemoryError
Heap	Yes	objects, instance vars	OutOfMemoryError
Java Stack	No (per thread)	call frames, locals	StackOverflowError
PC Register	No	current instruction	—
Native Stack	No	JNI method frames	StackOverflowError

MENTAL MODEL — SHARED OFFICE VS PERSONAL DESKS

Heap + Method Area
shared by everyone

Stack + PC
one private desk per thread

Trick: infinite recursion blows the **Stack** (too many frames); too many live objects blows the **Heap**.

C2 JIT Compiler vs Interpreter

IN SHORT

- **Interpreter**: executes bytecode line-by-line. Fast startup, slow on repeats.
- **JIT**: compiles "hot" (frequently-run) bytecode to native code. Slower start, much faster after warm-up.
- JVM starts interpreting everything, then JIT-compiles the hot spots as it detects them.

MENTAL MODEL — WARMING UP



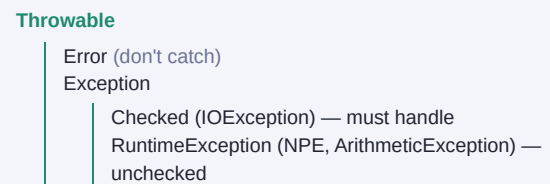
Trick: like a rehearsed speech — slow & careful the first time, fluent once memorized.

C2 Exception Handling

IN SHORT

- **Checked** (compile-time forced, e.g. `IOException`) vs **Unchecked/RuntimeException** (`NullPointerException`) vs **Error** (don't catch, e.g. `OutOfMemoryError`).
- try/catch/finally — finally always runs (cleanup).
- throws declares a checked exception a method might pass up.

MENTAL MODEL — THE HIERARCHY TREE



Trick: "Checked = compiler nags you. Unchecked = your bug. Error = not your job."

• JVM Tools quick reference

Tool	Purpose
<code>javap -c</code>	view bytecode instructions
<code>java -verbose:class</code>	trace class loading
<code>jps -l</code>	list running Java processes
<code>jcmd <pid></code>	JVM diagnostics (flags, heap, threads)
<code>jvisualvm</code>	GUI monitor: memory/CPU/threads/GC

Class 3 — Packages, Encapsulation & Access

Organizing code and controlling who can see what.

C3 Packages

IN SHORT

- A namespace container for classes — avoids name clashes (two different Teacher classes can coexist).
- package must be the first line in the file; lowercase by convention.
- Sub-packages are NOT auto-imported — import explicitly.

SNIPPET

```
school/math/Teacher.java    // package school.math
school/science/Teacher.java // package
school.science
// same class name, different package — no clash!
```

Trick: packages are folders + a namespace label rolled into one.

C3 Encapsulation

IN SHORT

- Bundle data + methods into a class; hide internal state behind private fields.
- Access only via public getters/setters — setters can **validate**.

SNIPPET

```
public void setAge(int age) {
    if (age > 0 && age < 150) this.age = age;
    // invalid data silently rejected, not just stored
}
```

Trick: "no field is public unless you want anyone able to set it to garbage."

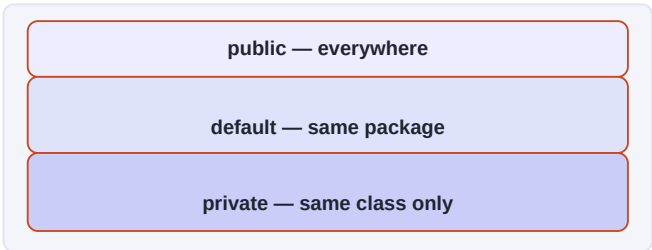
C3 Access Modifiers

VISIBILITY TABLE (PRIVATE → PUBLIC)

Visibility	private	default	public
Same class	YES	YES	YES
Same package	NO	YES	YES
Different package	NO	NO	YES

(protected = default + subclass access across packages — class 4.)

MENTAL MODEL — CONCENTRIC CIRCLES



Trick: start everything private, widen only when something outside genuinely needs it.

Practice (d): StudentReportCard — private fields, validated setters (0–100), getPercentage()/getGrade() derived methods, formatted printReportCard().

Class 4 — Inheritance & Polymorphism

Reusing and specializing behavior across classes.

• Inheritance

- `extends` — child gets parent's fields/methods. Single, multilevel ($A \rightarrow B \rightarrow C$), hierarchical (1 parent, many children) all supported.
- **No multiple inheritance of classes** — avoids the Diamond Problem (which parent's method wins?).

• Protected Access

- `protected` = default + visible to subclasses even in a **different package**.
- Use when children should touch parent fields directly, but randoms shouldn't.

• `this` & `super`

- `this.x = x` disambiguates field vs param; `this(...)` chains constructors.
- `super(...)` calls parent constructor — must be the **first** line; can't use both `this()` and `super()` together.

• Quick trick

- "Diamond problem" → why Java classes can't have 2 parents.
- "this = me, super = my parent." Constructors always call one or the other, first line, automatically if you don't.

C4 Polymorphism

IN SHORT

	Compile-time	Runtime
Achieved by	Overloading	Overriding
Resolved	compile time	runtime
Binding	early	late
Needs inheritance?	No	Yes

MENTAL MODEL

Overloading
same name, diff params
compiler decides

Overriding
same signature
JVM decides at runtime

Trick: "overLOADing = LOAD more versions (compile time).
overRIDing = RIDE over the parent's version (runtime)."

Practice (e): Employee payroll — FullTimeEmployee/PartTimeEmployee extends Employee, override calculatePay(), overload displayInfo(), store mixed types in an Employee[] and watch polymorphism pick the right method.

Class 5 — Abstraction & Object Relationships

Hiding complexity, and how objects relate to and own each other.

C5 Abstraction

IN SHORT

- Hide the **how**, expose only the **what**. Two tools: abstract classes (0–100%) and interfaces (100%).
- Abstract class: mixes abstract+concrete methods, has fields/constructors, but **can't be instantiated**.

```
abstract class Shape {  
    abstract double area();    // no body  
    void describe() { ... }    // concrete  
}
```

MENTAL MODEL — THE TV REMOTE



Trick: ask "does the caller need the HOW, or just the WHAT?" If just WHAT → abstract it.

C5 Interfaces

IN SHORT

- Pure contract: abstract methods by default; since Java 8 also default/static methods.
- No constructors, no instance fields (only static final). A class can implement **multiple** interfaces — solves the diamond problem.

	Abstract class	Interface
Fields	instance vars OK	static+final only
Multi-inherit	No	Yes

SNIPPET

```
interface Drawable {  
    void draw();    // abstract by default  
    double area();  
}  
class Circle implements Drawable { ... }
```

Trick: interface = "contract for unrelated classes"; abstract class = "shared code for close relatives."

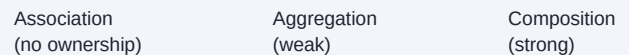
C5 Association, Aggregation & Composition

IN SHORT

	Association	Aggregation	Composition
Relation	"uses-a"	"has-a" weak	"has-a" strong
Child created	independent	outside, passed in	inside parent
Parent dies →	no effect	child survives	child dies too
Example	Driver ↔ Car	Library → Books	House → Rooms

```
// Aggregation: passed in from outside  
library.addBook(new Book(...));  
// Composition: created inside the parent  
class House { rooms.add(new Room(...)); }
```

MENTAL MODEL — THE OWNERSHIP SPECTRUM



Trick — the lifecycle test: "can the part exist without the whole?" YES+created-outside → aggregation. NO+created-inside → composition. Neither owns → association.

Practice (f): Airtribe LMS — abstract Learner (abstraction) → JavaLearner/NodeJSLearner (inheritance); Cohort owns its Learners (composition) but only references an Instructor (aggregation); Course ↔ Cohort (association).

Class 6 — Design Principles & SOLID

Rules of thumb for code that stays maintainable as it grows.

C6 Design Principles — the big picture

IN SHORT

- Guidelines for code that is **maintainable**, **scalable**, **reusable**, **robust**.
- Coming up: **DRY**, **KISS**, **YAGNI**, the 5 **SOLID** principles, composition-over-inheritance.

MENTAL MODEL — COMPASS VS. MAP

Principles
= compass (direction)

Patterns
= map route (specific path)

Trick: principles tell you *what to aim for*; patterns show you *how to get there*.

C6 DRY, KISS & YAGNI

IN SHORT

Principle	Rule	Ask yourself
DRY	one source of truth	"Have I written this before?"
KISS	prefer the plain solution	"Can a junior read this in 10s?"
YAGNI	build only what's needed now	"Does anyone need this today?"

```
// KISS: guard clauses beat nesting
if (x == null) return;
if (x <= 0) return;
work();
```

MENTAL MODEL — THREE GUARDRAILS

DRY
no dup logic

KISS
no needless complexity

YAGNI
no speculative code

Trick: "Don't copy, Keep calm, You ain't need it yet." Violations: copy-pasted tax logic, 6-level if-else, an unused multiply().

SOLID — five principles for maintainable OOP

S Single Responsibility Principle

IN SHORT

- "A class should have only **one reason to change**." — Uncle Bob

```
// good: split by responsibility
class OrderService { placeOrder(); }
class OrderCalculator { calculateTotal(); }
class EmailService { sendEmail(); }
```

MENTAL MODEL — THE RESTAURANT

Chef cooks

Waiter serves

Cashier bills

Trick: describe the class in one sentence. Needs "AND"? Split it.

O Open/Closed Principle

IN SHORT

- Open for **extension**, closed for **modification**. Kills the growing if-else chain.

```
interface Shape { void draw(); }
class Triangle implements Shape { draw(){...} }
// Circle, Rectangle never touched
```

MENTAL MODEL — THE USB PORT

USB port (never rewired)

← any device plugs in

Trick: "New variant? New class. Done."

L Liskov Substitution Principle

IN SHORT

- Any subclass must be usable wherever the parent is expected — no surprises.

```
abstract class Bird { abstract void move(); }
class Sparrow extends Bird { move(){fly();} }
class Ostrich extends Bird { move(){walk();} }
```

MENTAL MODEL — THE SWAP TEST

Expects Parent → Swap in Child → Still works?

Trick: a "driver" who can't drive manual broke the contract.

I Interface Segregation Principle

IN SHORT

- Don't force classes to depend on methods they don't use. Many small interfaces > one fat one.

```
interface Workable { void work(); }
class RobotWorker implements Workable {
    work() { ... } // no eat() forced on it
}
```

MENTAL MODEL — THE MENU

50-item menu (fat) → Veg / Non-veg menus

Trick: spot the smell — a method that's just throw new UnsupportedOperationException().

D Dependency Inversion Principle

IN SHORT

- High-level modules shouldn't depend on low-level ones directly — both depend on an abstraction. Enables **dependency injection**.

```
interface Switchable { void turnOn(); }
class Switch {
    Switch(Switchable d) { this.device = d; } // injected
}
```

MENTAL MODEL — INVERSION

High-level: Switch

↓ depends on ↓

Interface: Switchable

↑ implemented by ↑

Low-level: Bulb, Fan

Trick: USB-C charger — depend on the plug shape, not the brand.

+ Composition over Inheritance

IN SHORT

- Prefer "has-a" (compose behavior from parts) over "is-a" (inherit a rigid hierarchy).
- Avoids duplicated fields across siblings, the fragile-base-class problem, and lets behavior change **at runtime**.

```
class Duck {
    SwimBehavior swim; // HAS-A
    FlyBehavior fly; // mix & match per duck!
}
```

MENTAL MODEL — THE SWAPPABLE ENGINE

Car IS NOT an engine → Car HAS an engine (swap it!)

Use inheritance

true IS-A, shallow hierarchy

Use composition

behavior changes at runtime

Class 8 — Applying OOP Relationships

Taking association/aggregation/composition & composition-over-inheritance from theory to a real system.

C8 Deciding Association vs. Aggregation vs. Composition

IN SHORT — FROM THE FOOD DELIVERY SYSTEM BUILD

- **Restaurant ↔ MenuItem → Aggregation.** If the restaurant shuts down, "Butter Chicken" can still exist on another menu. Items are created outside and addMenuItem()-ed in.
- **Order ↔ OrderItem → Composition.** "2x Butter Chicken in Order #1001" is meaningless outside that order. The Order creates its OrderItems internally (constructor is package-private).
- **Customer ↔ Order and DeliveryAgent ↔ Order → Association.** Independent lifecycles, no ownership either way.

MENTAL MODEL — THE LITMUS QUESTION

"If the parent disappears tomorrow, does the child still make sense?"

YES, and it was passed in → Aggregation

NO, it was created inside → Composition

Neither owns the other → Association

Trick: two enums (Category, OrderStatus) instead of strings — compiler catches typos, autocomplete works.

Also in class 8: a_composition_over_inheritance re-runs the Class 6 principle against a concrete inheritance-gone-rigid problem; b_association_aggregation_composition re-demos all three relationships standalone before the practice problem combines them.

Practice (c): Online Food Delivery System — Restaurant, MenuItem, Customer, Order/OrderItem, DeliveryAgent, a PaymentMethod abstraction (CreditCardPayment/UPIPayment), a Trackable interface, and Category/OrderStatus enums — the relationships above, all in one system.

Class 9 — Creational Design Patterns

Proven, named solutions to "how should I create this object?"

C9 Intro to Design Patterns

IN SHORT

- A pattern is a proven, reusable **idea** (not code) with a **name** — shared vocabulary. From the "Gang of Four" (1994).
- **Principles** = grammar of good code. **Patterns** = sentences built with that grammar.

Family	Concerned with	Examples
Creational	how objects are created	Singleton, Builder, Factory
Structural	how objects compose	Decorator, Adapter
Behavioral	how objects interact	Observer, Strategy

MENTAL MODEL — THE DECISION MAP

Only **ONE** in the whole program? → Singleton

Lots of optional knobs? → Builder

Choice depends on runtime input? → Factory

Need a whole **MATCHED FAMILY**? → Abstract Factory

C9 Singleton

IN SHORT

- Ensure exactly **one instance**, one global access point: private constructor + private static field + public static `getInstance()`.
- Lazy init is **not thread-safe** alone — use double-checked locking for lazy + safe.

```
if (instance == null) {
    synchronized (Singleton.class) {
        if (instance == null) instance = new Singleton();
    }
}
```

MENTAL MODEL — THE SCHOOL PRINCIPAL

Many teachers, many students →

exactly **ONE** Principal

Trick: "One instance, private constructor, static field, static getter."
Real uses: Logger, config, connection pool.

C9 Builder

IN SHORT

- Fixes the "telescoping constructor" problem (too many positional args, easy to swap two booleans by mistake).
- Each setter returns this (fluent chaining); `.build()` validates and returns an **immutable** product.

```
new User.Builder("Alice", "a@x.com")
    .age(25).city("NY").build();
```

MENTAL MODEL — THE SUBWAY SANDWICH

Bread? Cheese? Veggies? →

answer what you care about →

"that's it" (build)

Trick: real uses — `StringBuilder`, `OkHttpClient.Builder`, `AlertDialog.Builder`.

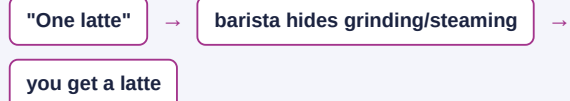
C9 Factory Method

IN SHORT

- Hide new behind a factory that decides the concrete class from input, so callers depend only on the **product interface**.
- Static factory (simple, switch-based) vs. Factory Method (abstract creator, subclass overrides) — latter needs zero changes to add a product.

```
Notifier n = NotifierFactory.create(channel);  
// caller never writes `new EmailNotifier()`
```

MENTAL MODEL — THE BARISTA



Trick: "Factory hides new. Caller names a product; factory decides the class."

C9 Abstract Factory

IN SHORT

- "Factory of factories" — creates a whole **family** of related products that must match (never a Mac button with a Windows checkbox).
- Client receives the factory via injection, never imports concrete types — adding a new family means zero client changes.

MENTAL MODEL — PHONE THEME



Trick: Factory Method makes **one** product; Abstract Factory makes a **matched set**, picked once.

Practice (e / warmup): Cloud Storage SDK (Singleton ConfigManager, Builder StorageRequest, Factory StorageClientFactory, Abstract Factory CloudFactory bundling client+logger per provider) and the "Slice of Heaven" pizza warmup (singleton shop settings, builder custom pizza, factory presets, abstract factory themed combos).

Structural & Behavioral Design Patterns

Five more GoF patterns, not tied to a specific class session in the source repo.

Observer

IN SHORT

- One-to-many: when the **Subject** changes state, all registered **Observers** get notified automatically.

```
void setPrice(double p) {  
    price = p;  
    for (Observer o : observers) o.update(name, price);  
}
```

MENTAL MODEL

Subject → Observer A, B, C

Trick: "Don't call us, we'll call you." Real use: GUI listeners, Kafka pub/sub. Pitfall: forgetting to unregister → memory leak.

Strategy

IN SHORT

- Define a family of interchangeable algorithms; **Context** delegates to whichever **Strategy** it's holding.

```
class ShoppingCart {  
    PaymentStrategy strategy;  
    void checkout(double t) { strategy.pay(t); }  
}
```

MENTAL MODEL

Context → Strategy A / B / C

Trick: "Same job, different ways to do it, pick one at runtime." Real use: Comparator<T>.

Chain of Responsibility

IN SHORT

- Pass a request along a chain of handlers; each either **processes** it or **passes it on**.

```
void handle(Ticket t) {  
    if (canHandle(t)) process(t);  
    else if (next != null) next.handle(t);  
}
```

MENTAL MODEL

Handler A → Handler B → Handler C

Trick: "Pass the buck until someone handles it." Real use: servlet filters, try/catch chains. Pitfall: no fallback → silently dropped request.

Decorator

IN SHORT

- Attach responsibilities to an object dynamically by **wrapping** it in same-interface layers — alternative to subclassing.

```
Coffee order = new WhipCreamDecorator(  
    new SugarDecorator(  
        new MilkDecorator(new SimpleCoffee())));
```

MENTAL MODEL

WhipCream(Sugar(Milk(Coffee)))

Trick: "wrap it like a gift, each layer adds something." Real use: BufferedReader(new InputStreamReader(...)). Watch: order can matter, identity changes.

State

IN SHORT

- Object changes behavior as its internal **State** changes; the state itself triggers transitions via the **Context**.

```
[Draft] --submit(--> [Review] --approve(-->
[Published]
```

STRATEGY VS. STATE

	Strategy	State
Who switches?	client, explicitly	state switches itself
Purpose	swap algorithm	model a lifecycle

Trick: "Context is the machine, each state is a mode." Real use: order lifecycle, TCP connection states.

Practice Questions — Design Patterns

2–3 questions per pattern — mix of "spot the problem," refactor, and design-from-scratch.

• Singleton

- Two threads might create your `Logger` simultaneously on first use. Which variant fixes this, and why does plain lazy init fail?
- A teammate gives `Singleton` a `public` constructor "just in case." What breaks, and how do you still allow tests to inject a mock?
- Design a `ConfigManager` singleton (region, provider, debug flag) using double-checked locking — write the field, constructor, and `getInstance()`.

• Factory Method

- Adding `SlackNotifier` to `EmailNotifier` / `SMSNotifier` : which files change with Static Factory vs. Factory Method (abstract creator)? Compare.
- Refactor an `if (type.equals("email")) ... else if ...` block repeated across five call sites into a factory.
- Write `PaymentHandlerFactory.create(String method)` for `"upi"/"card"/"paypal"` ; explain why callers should only ever see `PaymentHandler`.

• Observer

- A `Stock` notifies 200 observers per tick, but closed UIs never unregister. Name the bug and fix it.
- Compare polling ("check every second") to Observer — which is more efficient, and why?
- Design a `WeatherStation` notifying `PhoneDisplay` and `WebDashboard` on temperature change — sketch the interfaces and the notify loop.

• Chain of Responsibility

- Design a support-ticket chain: `Level1Handler` → `Level2Handler` → `Level3Handler`. Write `handle()`.
- What happens if a ticket reaches the end of the chain unhandled? How do you guard against that silently?
- vs. one long if-else for the same problem — what does the chain buy you when a new handler is added?

• State

- Model a `TrafficLight` (RED → GREEN → YELLOW → RED) — sketch the state interface and one transition.
- A junior dev writes `if (status.equals("RED")) ... else if ...` inside `TrafficLight` itself. Which principle does this violate, and how does State fix it?
- In one sentence, how does State differ from Strategy, using the Draft → Review → Published example?

• Builder

- Rewrite `new Pizza("large", "thin", true, List.of("olives"), 2, false)` using a Builder.
- Why should the built `Product` have no public setters? What breaks if it does?
- Design `StorageRequest.Builder` : required `bucket` / `key` in the constructor, optional `encrypted` / `contentType` chained, validation in `build()`.

• Abstract Factory

- Explain, with an example, why mixing `MacButton` with `WinCheckbox` is exactly the bug this pattern prevents.
- Design a `CloudFactory` for AWS/GCP/Azure, each producing a matched `StorageClient` + `Logger`. Sketch the interface and one concrete factory.
- When would plain Factory Method be enough, and Abstract Factory overkill? Give a one-sentence rule.

• Strategy

- `ShoppingCart.checkout()` has a growing `if (type.equals("credit")) ...` chain. Refactor it with Strategy.
- What's the key difference between Strategy and Factory? (One creates objects; the other swaps ...?)
- Design a `SortingContext` that switches between `QuickSortStrategy` and `MergeSortStrategy` at runtime.

• Decorator

- From `SimpleCoffee`, write the decorator stack for "milk + sugar + whipped cream" and its `getCost()`.
- Why does `instanceof SimpleCoffee` break after wrapping it? What does that imply about relying on concrete types?
- Name a JDK class that uses Decorator and identify which role (Component/ConcreteComponent/Decorator) it plays.

Appendix — Practice Problems Index

Class	Problem	Concepts practiced
1 (s10)	Airtribe Learner Manager (console)	arrays, Scanner, switch, if-else validation, while-loop menu
2 (s11)	Learner Manager, revisited	same LMS exercise, reinforcing fundamentals after JVM internals
3 (d)	Student Report Card System	encapsulation, validated setters, derived getPercentage()/getGrade()
4 (e)	Employee Payroll System	inheritance, super() chaining, method overriding & overloading, polymorphic array
5 (f)	Airtribe LMS (OOP)	abstraction, interfaces, encapsulation, inheritance, composition, aggregation, association — all together
8 (c)	Online Food Delivery System	choosing association vs. aggregation vs. composition deliberately; enums; Trackable interface; payment abstraction
9 (e)	Cloud Storage SDK	Singleton config, Builder request, Factory client, Abstract Factory client+logger family
9 (warmup)	"Slice of Heaven" Pizza App	all 4 creational patterns applied to one small, concrete domain